Téma 5: Funkce (Definice, volání a rozsah)

Teoretický základ

Funkce jsou zapouzdřené bloky kódu, které se provedou jen ve chvíli, kdy jsou zavolány.

- Základ: Klíčové slovo `def`. Funkce může mít vstupní parametry a může vracet výsledek pomocí slova `return`. (Důležité: `print()` výsledek na obrazovku jen vypíše, ale `return` ho předá programu k dalšímu počítání).
- **Parametry vs. Argumenty:** Parametr je název proměnné v definici funkce (např. `def mocnina(x):`). Argument je konkrétní hodnota předaná při volání (např. `mocnina(5)`).
- **Rozsah platnosti (Scope):** Proměnná vytvořená uvnitř funkce "umírá" při skončení funkce (lokální proměnná). Neovlivňuje proměnné venku.

Základní koncept k zamyšlení: Výchozí parametry

Pokud v definici funkce uvedeme hodnotu, jedná se o tzv. klíčový (výchozí) argument: `def pozdrav(jmeno="Host"):`. Pokud tuto funkci zavoláme jednoduše jako `pozdrav()` bez jakýchkoliv hodnot v závorce, program nespadne, ale automaticky dosadí slovo "Host".

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Napište funkci `vypocti_obsah_obdelniku(a, b)`, která přijme strany obdélníku a vrátí (přes `return`) jeho obsah. Venku (v hlavním těle programu) funkci zavolejte, výsledek uložte do proměnné a vypište.

Řešení úkolu 1

```
def vypocti_obsah_obdelniku(a, b):
    obsah = a * b
    # Funkce si výsledek nesmí nechat pro sebe, musí jej vrátit (return)
    return obsah

# Správné volání a uložení návratové hodnoty do vnější proměnné
vysledek = vypocti_obsah_obdelniku(5, 10)
print(f"Obsah obdélníku je: {vysledek}")
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Vytvořte funkci `pocet_samohlasek(text)`, která jako parametr přijme libovolný řetězec a vrátí počet samohlásek (a, e, i, o, u), které se v něm nacházejí. Ošetřete funkci tak, aby správně počítala i v případě, že uživatel použije velká písmena.

Řešení úkolu 2

```
def pocet_samohlasek(text):
    pocitadlo = 0
    vzory_samohlasek = "aeiouyáéíóúůý"

    # Metoda lower() zajistí, že vše v textu převedeme na malá písmena.
    for znak in text.lower():
        if znak in vzory_samohlasek:
            pocitadlo += 1

    return pocitadlo

# Otestování funkce
print(pocet_samohlasek("Ahoj Světe!")) # Výstup: 4
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Vytvořte funkci `faktorial(n)`. Faktoriál čísla (značený vykřičníkem) je výsledek násobení všech čísel od 1 do n (např. 4! = 4 * 3 * 2 * 1). Zkuste funkci napsat **bez for cyklu pomocí rekurze** – tedy situace, kdy funkce uvnitř sebe volá samu sebe.

Řešení úkolu 3

```
# Řešení pomocí rekurze
def faktorial(n):
    # Ukončovací podmínka (tzv. Base Case). Je kritický důležitá,
    # bez ní by funkce do nekonečna volala samu sebe a program by spadl.
    if n == 1 or n == 0:
        return 1

    # Hodnotu 'n' vynásobíme výsledkem zavolání téže funkce o jedna menší
    return n * faktorial(n - 1)

print(f"Faktoriál 5! je roven: {faktorial(5)}")
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: Vytvořte funkci nazvanou `pozdrav(jmeno)`, která po svém zavolání vypíše: *"Dobrý den, [jmeno]."* Navrhněte parametr tak, aby byl výchozí. Pokud tedy zavoláte prázdnou závorku `pozdrav()`, vypíše se automaticky *"Dobrý den, Neznámý uživateli."*

Řešení úkolu 4

```
# Parametr získává záchrannou hodnotu, tzv. 'Default argument'
def pozdrav(jmeno="Neznámý uživateli"):
    print(f"Dobrý den, {jmeno}.")

# Volání s argumentem
pozdrav("Karlo")  # Vypíše: Dobrý den, Karlo.

# Volání bez argumentu nespadne na chybě, použije se výchozí:
pozdrav()         # Vypíše: Dobrý den, Neznámý uživateli.
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: Napište logickou funkci `je_sude(cislo)`. Funkce přijme jedno celé číslo a vrátí (přes `return`) hodnotu `True`, pokud je číslo dělitelné beze zbytku dvěma. V opačném případě vrátí `False`. Vyřešte to uvnitř funkce na jeden jediný řádek (nevyužívejte rozsáhlý if/else).

Řešení úkolu 5

```
def je_sude(cislo):
    # Výraz 'cislo % 2 == 0' je sám o sobě boolean hodnota (vyhodnotí
    # se jako True nebo False). Můžeme ji proto rovnou celou vrátit.
    return cislo % 2 == 0

# Testování
print("Je 10 sudé?", je_sude(10))  # True
print("Je 7 sudé?", je_sude(7))    # False
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: (Použití `**kwargs`). Někdy potřebujeme vytvořit tak flexibilní funkci, která umí přijmout libovolný počet pojmenovaných vlastností naráz (např. `jmeno="Jan", vek=20, obor="IT"`). Definujte funkci `profil_studenta` pomocí dvou hvězdiček, vytiskněte tyto vlastnosti na obrazovku hezky pod sebe.

Řešení úkolu 6

```
# Dvě hvězdičky (kwargs = Keyword Arguments) shromáždí
# všechny pojmenované vstupy z volání do jednoho Python slovníku
(dictionary).
def profil_studenta(**kwargs):
    print("--- PROFIL STUDENTA ---")

    # Projdeme klíče a hodnoty získaného slovníku a vypíšeme je
    for klic, hodnota in kwargs.items():
        print(f"{klic.capitalize()}: {hodnota}")

# Můžeme libovolně volat, funkce zvládne jakoukoliv šířku argumentů
profil_studenta(jmeno="Karel", vek=21, univerzita="CVUT")
```